

软件工程基础知识

历年真题考情：

本章节每年会考10-12分左右。

第二版更新：

第二版教材里对应第5章，相关概念和定义改动的比较多，尤其是对于软件工程阶段的划分，传统的软件设计的阶段划分等，新教材里都删除了，也缺少了测试用例设计、运行维护阶段内容，这会导致大家做历年真题感觉陌生，这块的话我们会给大家按最新教材内容录制，同时通过每年最新直播讲题来讲往年真题的老的知识点，本章节对应5.7项目管理单独独立为一个章节，不在此赘述。

软件工程概述
软件过程模型
能力成熟度模型
逆向工程
需求工程
结构化分析与设计
测试基础知识
系统运行与维护
净室软件工程
基于构件的软件工程

文老师软考教育

软件工程概述

文老师软考教育

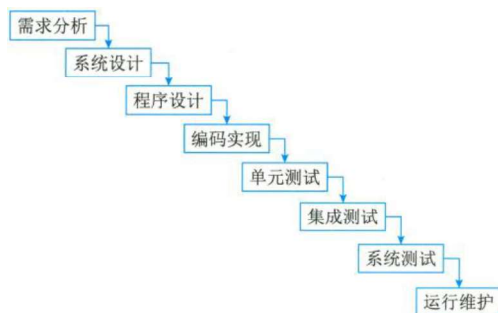
◆软件工程过程是指为获得软件产品，在软件工具的支持下由软件工程师完成的一系列软件工程活动，包括以下4个方面。

- (1) P (Plan)——**软件规格说明**。规定软件的功能及其运行时的限制。
- (2) D (Do)——**软件开发**。开发出满足规格说明的软件。
- (3) C (Check)——**软件确认**。确认开发的软件能够满足用户的需求。
- (4) A (Action)——**软件演进**。软件在运行过程中不断改进以满足客户新的需求。

◆软件要经历从**需求分析**、**软件设计**、**软件开发**、**运行维护**，直至被**淘汰**这样的全过程，这个全过程称为软件的生命周期。软件生命周期描述了软件从生到死的全过程。

◆为了使软件生命周期中的各项任务能够有序地按照规程进行，**需要一定的工作模型对各项任务给予规程约束**，这样的工作模型被称为**软件过程模型**，有时也称之为软件生命周期模型。

软件过程模型



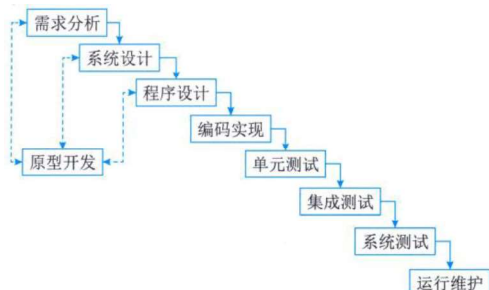
◆瀑布模型（Waterfall Model）是最早使用的软件过程模型之一，包含一系列活动。这些活动从一个阶段到另一个阶段逐次下降，它的工作流程在形式上很像瀑布，因此被称为瀑布模型，如图所示：

◆瀑布模型的特点是因果关系紧密相连，前一个阶段工作的输出结果，是后一个阶段工作的输入。每一个阶段都是建筑在前一个阶段正确实施的结果之上。每一个阶段工作完成后都伴随着一个里程碑（一组检查条件），对该阶段的工作进行审查和确认。

◆瀑布模型的主要缺点：

- （1）软件需求的完整性、正确性等很难确定，甚至是不可能和不现实的。
- （2）瀑布模型是一个严格串行化的过程模型，使得用户和软件项目负责人要相当长的时间才能得到一个可以看得见的软件系统。
- （3）瀑布模型的基本原则是在每个阶段一次性地完全解决该阶段的工作，不会出现遗漏、错误等情况，而实际上这是不现实或不可能的。

软件过程模型



◆原型模型（Prototype Model）又称快速原型。该模型如图所示：原型模型主要有以下两个阶段：

（1）原型开发阶段。即快速开发一个原型；开发原型可以考虑3种途径：利用模拟软件系统的人机界面和人机交互方式；真正开发一个原型；找来一个或几个正在运行的类似软件进行比较。

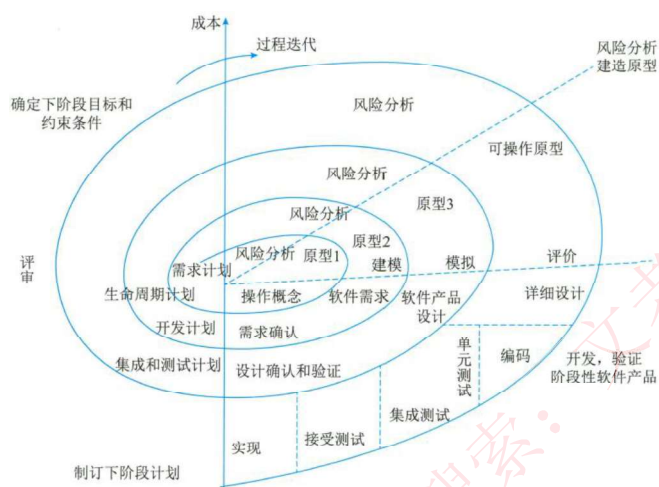
（2）目标软件开发阶段。在征求用户对原型的意见后对原型进行修改完善，确认软件系统的需求并达到一致的理解，进一步开发实际系统。

◆原型模型的使用应该注意以下内容。

- 用户对系统的认识模糊不清，无法准确回答目标系统的需求。
- 要有一定的开发环境和工具支持。
- 经过对原型的若干次修改，应收敛到目标范围内，否则可能会失败。
- 对大型软件来说，原型可能非常复杂而难以快速形成，如果没有现成的原型模型，就不应考虑用原型法。

◆按照原型的作用不同，出现了抛弃型原型和演化性原型。抛弃型原型是将原型作为需求确认的手段，在需求确认结束后，原型就被抛弃不用，重新采用一个完整的瀑布模型进行开发。演化性原型是在需求确认结束后，不断补充和完善原型，直至形成一个完整的产品。

软件过程模型



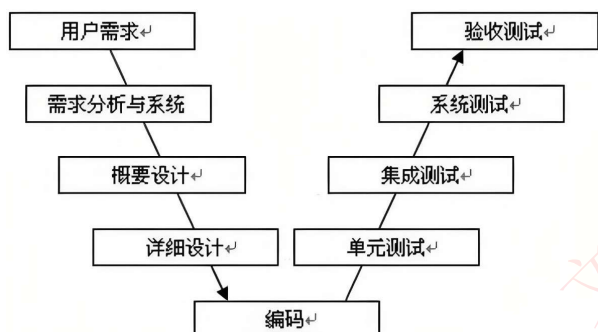
◆螺旋模型（Spiral Model）是在快速原型的基础上扩展而成。实际上，它是生命周期模型与原型模型的结合，如图所示。

◆这种模型把整个软件开发流程分成多个阶段，每一个阶段都由4部分组成，它们是：目标设定、风险分析、开发和有效性验证、评审。

◆螺旋模型的软件开发过程实际是上述4个部分的迭代过程，每迭代一次，螺旋线就增加一圈，软件系统就生成一个新版本，这个新版本实际上是对目标系统的一个逼近。

◆该模型支持大型软件开发，适用于面向规格说明、面向过程和面向对象的软件开发方法，也适用于几种开发方法的组合。

软件过程模型



◆V模型从整体上看，就是一个V字型的结构，由左右两边组成。左边的下画线分别代表了需求分析、概要设计、详细设计、编码。右边的上画线代表了单元测试、集成测试、系统测试与验收测试。V模型的特点如下：

- (1) 单元测试的主要目的是针对编码过程中可能存在的各种错误；
- (2) 集成测试的主要目的是针对详细设计中可能存在的问题；
- (3) 系统测试主要针对概要设计，检查系统作为一个整体是否有效地得到运行；
- (4) 验收测试通常由业务专家或者用户进行，以确认产品能真正符合用户业务上的需要。
- (5) V模型用于需求明确和需求变更不频繁的情形。

◆特别注意：测试阶段依赖的文件是对应开发阶段的上一个阶段，如单元测试的依据是详细设计文件。

软件过程模型

◆增量模型：首先开发核心模块功能，而后与用户确认，之后再开发次核心模块的功能，即每次开发一部分功能，并与用户需求确认，最终完成项目开发，优先级最高的服务最先交付。由于并不是从系统整体角度规划各个模块，因此不利于模块划分。难点在于如何将客户需求划分为多个增量。与原型不同的是增量模型的每一次增量版本都可作为独立可操作的作品，而原型的构造一般是为了演示。

◆喷泉模型：是一种以用户需求为动力，以对象作为驱动力的模型，适合于面向对象的开发方法。使开发过程具有迭代性和无间隙性。

◆基于构件的开发模型CBSD：利用预先包装的构件来构造应用系统。构件可以是组织内部开发的构件，也可以是商品化成品软件构件。

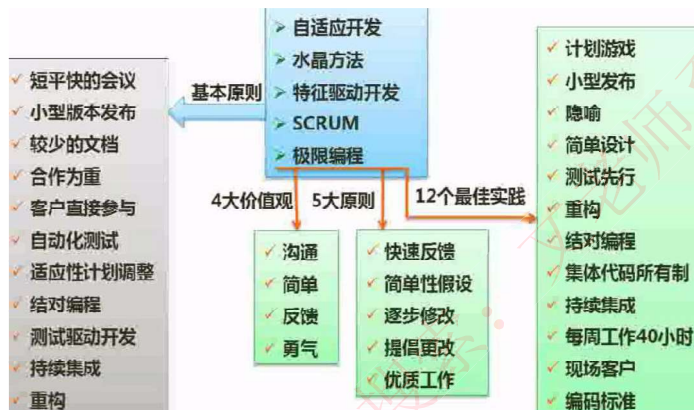
◆特点是增强了复用性，在系统开发过程中，会构建一个构件库，供其他系统复用，因此可以提高可靠性，节省时间和成本。

◆形式化方法模型：建立在严格数学基础上的一种软件开发方法，主要活动是生成计算机软件形式化的数学规格说明。

软件过程模型

◆敏捷模型

开发宣言：个体和交互 胜过 过程和工具、可以工作的软件 胜过 面面俱到的文档、客户合作 胜过 合同谈判、响应变化 胜过 遵循计划。



◆敏捷方法区别于其他方法的两个特点：

- (1) 是“适应性”而非“预设性”。
- (2) 是“面向人的”而非“面向过程的”。

◆敏捷方法的核心思想：

- (1) 敏捷方法是适应型，而非可预测型。拥抱变化，适应变化。
- (2) 敏捷方法是以人为本，而非以过程为本。发挥人的特性。
- (3) 迭代增量式的开发过程。以原型开发思想为基础，采用迭代增量式开发，发行版本小型化。

软件过程模型

◆主要敏捷方法：

(1) 极限编程 (XP)。基础和价值观是**交流、朴素、反馈和勇气**，即任何一个软件项目都可以从4个方面入手进行改善：**加强交流：从简单做起；寻求反馈；勇于实事求是。**

◆ XP 是一种**近螺旋式**的开发方法，它将**复杂的开发过程分解为一个一个相对比较简单的小周期**：通过积极的交流、反馈以及其他一系列的方法，开发人员和客户可以非常清楚开发进度、变化、待解决的问题和潜在的困难等，并根据实际情况及时地调整开发过程。

◆ XP提倡**测试先行**，为了将以后出现bug的几率降到最低。

(2) 水晶系列方法。都有**以人为中心**的理念，但在实践上有所不同。其目的是发展一种提倡“机动性的”方法，包含具有共性的核心元素，**每个都含有独特的角色、过程模式、工作产品和实践。**

(3) 并列争球法 (Scrum)。该方法侧重于**项目管理**。Scrum是**迭代式增量软件开发过程**，通常用于**敏捷软件开发**。Scrum包括了一系列实践和预定义角色的过程骨架（是一种流程、计划、模式，用于有效率地开发软件）。

◆在Scrum 中，使用**产品Backlog** 来管理产品的需求，产品Backlog 是一个**按照商业价值排序的需求列表**。根据Backlog的内容，将**整个开发过程被分为若干个短的迭代周期 (Sprint)**。在Sprint 中，Scrum 团队从产品Backlog 中挑选最高优先级的需求组成Sprint backlog。在每个迭代结束时，Scrum 团队将递交潜在可交付的产品增量。当所有Sprint 结束时，团队提交最终的软件产品。

(4) 特性驱动开发方法 (FDD)。是一个**迭代**的开发模型。认为有效的软件开发需要3个要素：**人、过程和技术**。有5个核心过程：**开发整体对象模型、构造特征列表、计划特征开发、特征设计和特征构建。**

软件过程模型

◆统一过程模型 (RUP)

◆ RUP 描述了**如何有效地利用商业的、可靠的方法开发和部署软件**，是一种**重量级过程**。RUP 类似一个在线的指导者，它可以**为所有方面和层次的程序开发提供指导方针、模版以及事例支持**。

◆ RUP 软件开发生命周期是一个**二维的软件开发模型**，RUP 中有**9个核心工作流**，如下：

·**业务建模**：理解待开发系统所在的机构及其商业运作，确保所有参与人员对待开发系统所在的机构有共同的认识，评估待开发系统对所在机构的影响。

·**需求**：定义系统功能及用户界面，使客户知道系统的功能，使开发人员理解系统的需求，为项目预算及计划提供基础。

·**分析与设计**：把需求分析的结果转化为分析与设计模型。

·**实现**：把设计模型转换为实现结果，对开发的代码做单元测试，将不同实现人员开发的模块集成为可执行系统。

·**测试**：检查各子系统之间的交互、集成，验证所有需求是否均被正确实现，对发现的软件质量上的缺陷进行归档，对软件质量提出改进建议。

·**部署**：打包、分发、安装软件，升级旧系统；培训用户及销售人员进行，并提供技术支持。

·**配置与变更管理**：跟踪并维护系统开发过程中产生的所有制品的完整性和一致性。

·**项目管理**：为软件开发项目提供计划、人员分配、执行、监控等方面的指导，为风险管理提供框架。

·**环境**：为软件开发机构提供软件开发环境，即提供过程管理和工具的支持。

软件过程模型

◆ RUP 把软件开发生命周期划分为**多个循环**，每个循环生成产品的一个新的版本，**每个循环依次由4个连续的阶段组成**，每个阶段完成确定的任务。这4个阶段如下。

- 初始阶段**：定义最终产品视图和业务模型，并**确定系统范围**。
- 细化阶段**：设计及确定**系统的体系结构**，制订工作计划及资源要求。
- 构造阶段**：**构造产品并继续演进需求**、体系结构、计划直至产品提交。
- 移交阶段**：把产品**提交给用户使用**。

◆ RUP 中定义了如下一些**核心概念**，理解这些概念对于理解RUP 很有帮助。

- 角色**：**Who 的问题**。角色描述某个人或一个小组的行为与职责。RUP预先定义了很多角色，如体系结构师、设计人员、实现人员、测试员和配置管理人员等，并对每一个角色的工作和职责都做了详尽的说明。
- 活动**：**How 的问题**。活动是一个有明确目的的独立工作单元。
- 制品**：**What的问题**。制品是活动生成、创建或修改的一段信息。
- 工作流**：**When 的问题**。工作流描述了一个有意义的连续的活动序列，每个工作流产生一些有价值的产品，并显示了角色之间的关系。

软件过程模型

◆ RUP的**特点**：

- (1) **用例驱动**：需求分析、设计、实现和测试等活动都是用例驱动的。
- (2) **以体系结构为中心**：包括**系统的总体组织和全局控制、通信协议、同步、数据存取、给设计元素分配功能、设计元素的组织、物理分布、系统的伸缩性和性能**等。是一个多维的结构，会采用多个视图来描述。在典型的4+1视图模型中：

- ◆ **分析人员和测试人员**关心的是系统的行为，会侧重于用例视图；
- ◆ **最终用户**关心的是系统的功能，会侧重于逻辑视图；
- ◆ **程序员**关心的是系统的配置、装配等问题，会侧重于实现视图；
- ◆ **系统集成人员**关心的是系统的性能、可伸缩性、吞吐率等问题，会侧重于进程视图；
- ◆ **系统工程师**关心的是系统的发布、安装、拓扑结构等问题，会侧重于部署视图。



- (3) **迭代与增量**。把整个项目开发分为**多个迭代过程**。在每次迭代中，只考虑系统的一部分需求，进行分析、设计、实现、测试和部署等过程；每次迭代是在已完成部分的基础上进行的，每次增加一些新的功能实现，以此进行下去，直至最后项目的完成。

软件能力成熟度模型

◆软件能力成熟度模型CMM

对软件组织化阶段的描述，随着软件组织定义、实施、测量、控制和改进其软件过程，软件组织的能力经过这些阶段逐步提高。针对软件研制和测试阶段。分为如下五个级别：

能力等级	特点
初始级 (Initial)	软件过程的特点是 杂乱无章 ，有时甚至很混乱，几乎没有明确定义的步骤，项目的成功 完全依赖个人的努力和英雄式核心人物的作用 。
可重复级 (Repeatable)	建立了 基本的项目管理过程和实践 来跟踪项目费用、进度和功能特性，有 必要的过程准则来重复 以前在同类项目中的成功。
已定义级 (Defined)	管理和工程两方面的 软件过程已经文档化、标准化，并综合成整个软件开发组织的标准软件过程 。所有项目都采用根据实际情况修改后得到的 标准软件过程来发和维护软件 。
已管理级 (Managed)	制定了软件过程和产品质量的 详细度量标准 。对软件过程和产品质量有定量的理解和控制。
优化级 (Optimized)	加强了定量分析，通过来自过程质量反馈和来自新观念、新技术的反馈使过程能 不断持续地改进 。

软件能力成熟度模型

◆能力成熟度模型集成CMMI

主要用于指导**软件开发过程的改进和进行软件开发能力的评估**。CMMI 提供了一个软件能力成熟度的框架，它将软件过程改进的步骤组织成**5个成熟度等级**，共包括18个关键过程域，52个过程目标，3168种关键时间，它为软件过程不断改进奠定了一个循序渐进的基础。五个成熟度级别如下：

能力等级	特点
初始级	过程通常是 随意且混乱 的。这些组织的成功依赖于组织内人员的能力与英雄主义
已管理级	组织要 确保策划、文档化、执行、监督和控制项目级的过程 ，并且需要为过程建立明确的目标，并能实现成本、进度和质量目标等。
已定义级	企业能够 根据自身的特殊情况定义适合自己企业和项目的标准流程 ，将这套管理体系与流程予以制度化，同时企业开始进行项目积累，企业资产的收集。
量化管理级	组织建立了 产品质量、服务质量以及过程性能的定量目标 。成熟度级别3级与4 级的关键区别在于 对过程性能的可预测 。
优化级	企业的项目管理达到了 最高的境界 。成熟度级别5级关注于通过增量式的与创新式的过程与技术改进，不断地改进过程性能。处于成熟度5级时，组织使用从多个项目收集来的数据对整体的组织级绩效进行关注。

逆向工程

◆**逆向工程**：软件的逆向工程是**分析程序**，力图在比源代码更高抽象层次上建立程序的表示过程，逆向工程是**设计的恢复过程**。逆向工程的四个级别：

实现级：包括程序的抽象语法树、符号表、过程的设计表示。

结构级：包括反映程序分量之间相互依赖关系的信息，例如调用图、结构图、程序和数据结构。

功能级：包括反映程序段功能及程序段之间关系的信息，例如数据和控制流模型。

领域级：包括反映程序分量或程序诸实体与应用领域概念之间对应关系的信息，例如E-R模型。

其中，**领域级抽象级别最高，完备性最低，实现级抽象级别最低，完备性最高。**

◆与逆向工程相关的概念有**重构、设计恢复、再工程和正向工程**。

(1) 重构是指在**同一抽象级别上转换系统描述形式**。

(2) 设计恢复指借助工具**从已有程序中抽象出有关数据设计、总体结构设计和过程设计**等方面的信息。

(3) 再工程是指在**逆向工程所获得信息的基础上，修改或重构已有的系统，产生系统的一个新版本**。

再工程是对**现有系统的重新开发过程**，包括**逆向工程、新需求的考虑过程和正向工程**三个步骤。它不仅能从已存在的程序中重新获得设计信息，而且还能使用这些信息来重构现有系统，以改进它的综合质量。在利用再工程重构现有系统的同时，一般会**增加新的需求**，包括增加新的功能和改善系统的性能。

(4) 正向工程是指不仅从现有系统中恢复设计信息，**而且使用该信息去改变或重构现有系统**，以改善其整体质量。

考试真题

CMMI有阶段式和连续式两个不同划分方式，其中阶段式共分为（）个阶段

A.3 B.4 C.5 D.6

答案：C

软件的逆向工程是分析程序，力图在比源代码更高抽象层次上建立程序的表示过程，逆向工程有四个级别，其中，使用用户指导下的搜索与变换方法可导出以下哪两个层级（）

A.实现级、功能级 B.功能级、领域级 C.实现级、结构级 D.结构级、领域级

答案：B

以下（）开发模型增加了风险评估的环节

A.瀑布模型 B.螺旋模型 C.V模型 D.增量模型

答案：B

在Scrum 中，使用产品Backlog 来管理产品的需求，产品Backlog 是一个按照商业价值排序的需求列表。

A实现难易度 B商业价值 C 最后期限 D技术成熟度

答案：B

需求工程

◆软件需求目前并没有统一的定义，但都包含以下几方面的内容。

- (1) 用户解决问题或达到目标所需条件或权能。
- (2) 系统或系统部件要满足合同、标准、规范或其他正式规定文档所需具有的条件或权能。
- (3) 一种反映上面(1)或(2)所述条件或权能的文档说明。它包括功能性需求及非功能性需求，非功能性需求对设计和实现提出了限制，比如性能要求、质量标准或者设计限制。

◆软件需求包括3个不同的层次：业务需求、用户需求和功能需求(也包括非功能需求)。

◆业务需求：反映了组织机构或客户对系统、产品高层次的目标要求。

◆用户需求：描述了用户使用产品必须要完成的任务，是用户对该软件产品的期望。这两种构成了用户原始需求文档的内容。

◆功能需求：规定了开发人员必须在系统中实现的软件功能，用户利用这些功能来完成任务，满足业务需要。所谓特性是指逻辑上相关的功能需求的集合，给用户处理提供能力并满足业务需求。

◆作为补充，软件需求规格说明还应包括非功能需求，它描述了系统展现给用户的行为和执行的操作等。它包括产品必须遵从的标准、规范和合约；外部界面的具体细节；性能要求；设计或实现的约束条件及质量属性。所谓约束是指对开发人员在软件产品设计和构造上的限制，常见的有设计约束和过程约束。质量属性是通过多种角度对产品的特点进行描述，从而反映产品功能。多角度描述产品对用户和开发人员都极为重要。

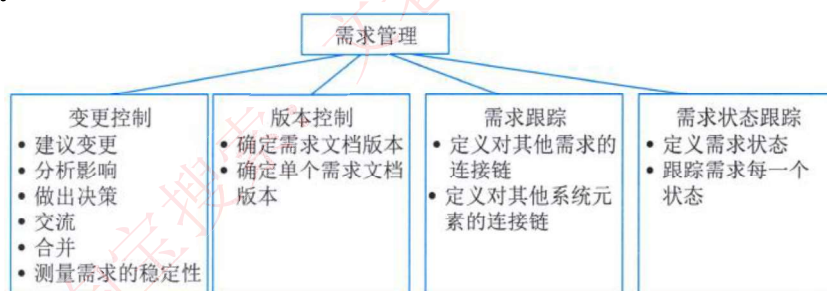
需求工程

◆需求工程是指应用已证实有效的原理、方法，通过合适的工具和记号，系统地描述待开发系统及其行为特征和相关约束。需求工程覆盖了体系结构设计之前的各项开发活动，主要包括分析客户要求、对未来系统的各项功能性及非功能性需求进行规格说明。需求工程的目标简单明了：确定客户需求，定义设想中系统的所有外部特征。

◆需求工程的活动：需求获取、需求分析、形成需求规格(需求文档化)、需求确认与验证、需求管理。

◆软件需求开发的最终文档经过评审批准后，则定义了开发工作的需求基线。这个基线在客户和开发者之间构筑了计划产品功能需求和非功能需求的一个约定。需求约定是需求开发和需求管理之间的桥梁。

◆需求管理是一个对系统需求变更、了解和控制的过程。需求管理过程与需求开发过程相互关联，当初始需求导出的同时就启动了需求管理规划，一旦形成了需求文档的初稿，需求管理活动就开始了。需求管理的主要活动如图所示。



需求获取

◆需求获取是开发者、用户之间为了定义新系统而进行的交流，需求获取是获得系统必要的特征，或者是获得用户能接受的、系统必须满足的约束。

◆需求获取参考步骤：

- 1) **开发高层的业务模型**：建立一个业务模型，描述用户的业务过程，确定用户的初始需求。
- 2) **定义项目范围和高层需求**：项目范围描述系统的边界以及系统与系统交互的参与者之间的关系。高层需求不涉及过多的细节，主要表示系统需求的概貌。常见的建模手段包括系统上下文图和系统顶层用例图等。
- 3) **识别用户角色和用户代表**。用户角色可以是人，也可以是与系统打交道的其他应用程序或硬件部件。如果是其他应用程序或硬件部件，则需要以熟悉这些系统或硬件的人员作为用户代表。
- 4) **获取具体的需求**。获取每个涉众的具体、完整和详细的需求。
- 5) **确定目标系统的业务 workflow**。具体到当前待开发的应用系统，确定系统的业务 workflow 和主要的业务规则。
- 6) **需求整理与总结**。最后对上面步骤取得的需求资料进行整理和总结，确定对软件系统的综合要求，即软件的需求。

需求获取

◆常见的需求获取法包括：

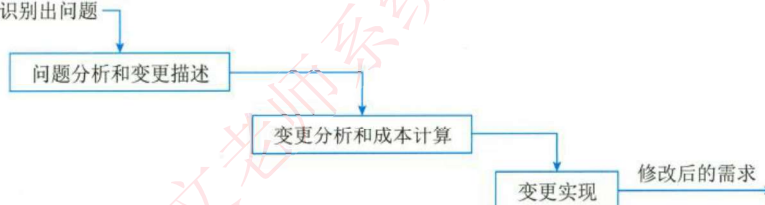
- (1) **用户面谈**：1对1-3，有代表性的用户。其形式包括**结构化和非结构化**两种。面谈过程需要认真的计划和准备；面谈之后，需要复查笔记的准确性、完整性和可理解性；把所收集的信息转化为适当的模型和文档；确定需要进一步澄清的问题。
- (2) **需求专题讨论会**：将相关涉众集中在一起集体讨论，与会者可以在应用需求上达成共识，对操作过程尽快取得统一的意见。参加会议的人员包括主持人、用户、技术人员、项目组人员。
- (3) **问卷调查**：用户多，无法一一访谈。可用于确认假设和收集统计倾向数据。
- (4) **现场观察**：主要是通过观察用户实际执行业务的过程，来直观地了解业务的执行过程，全面了解需求细节。执行业务可能是手工操作，也可能是在原有的业务系统上执行。
- (5) **原型化方法**：在需求的早期，用户往往在具体的需求定义上存在很多不确定性。此时往往可以通过在需求阶段采用原型化方法，通过开发系统原型以及用户的多次迭代反馈，解决在早期阶段需求不确定的问题，尤其是在人机界面等高度不确定的需求。
- (6) **头脑风暴法**：在一些新业务拓展的软件项目中，由于业务是新出现的，而且业务流程存在高度的不确定性，例如互联网上的新业务系统、App等，一群人围绕该业务，发散思维，不断产生新的观点，参会者敞开思想使各种设想在相互碰撞中激起大脑的创造性风暴，从而确定具体的需求。

需求变更和需求追踪

◆为了使开发组织能够**严格控制软件项目**，应该确保以下事项：

- 仔细**评估已建议的变更**。
- 挑选合适的人选**对变更做出判定**。
- 变更应及时**通知所有相关人员**。
- 项目要**按一定的程序来采纳需求变更**，对变更的过程和状态进行控制。

◆变更控制过程如图所示：识别出问题



(1) **问题分析和变更描述**。当提出一份变更提议后，需要对该提议做进一步的问题分析，检查它的有效性，从而产生一个更明确的需求变更提议。

(2) **变更分析和成本计算**。当接受该变更提议后，需要对需求变更提议进行影响分析和评估。变更成本计算应该包括对该变更所引起的所有改动的成本，例如修改需求文档、相应的设计、实现等工作成本。一旦分析完成并且被确认，应该进行是否执行这一变更的决策。

(3) **变更实现**。当确定执行该变更后，需要根据该变更的影响范围，按照开发的过程模型执行相应的变更。

需求变更和需求追踪

◆**变更控制委员会(CCB)**是项目所有者权益代表，负责**裁定接受哪些变更**。CCB由项目所涉及的多方成员共同组成，通常包括用户和实施方的决策人员。**CCB是决策机构，不是作业机构**，通常CCB的工作是通过评审手段来决定项目是否能变更，但**不提出变更方案**。

◆变更控制委员会应该有一个总则，用于描述变更控制委员会的目的、授权范围、成员构成、做出决策的过程及操作步骤。过程及操作步骤为：**制定决策、交流情况、重新协商约定**。

◆需求跟踪提供了由**需求到产品实现整个过程范围的明确查阅的能力**。需求跟踪的目的是建立与维护“需求—设计—编程—测试”之间的一致性，确保**所有的工作成果符合用户需求**。

◆需求跟踪有两种方式：

- (1) **正向跟踪**。检查《产品需求规格说明书》中的每个需求是否都能在后继工作成果中找到对应点。
- (2) **逆向跟踪**。检查设计文档、代码、测试用例等工作成果是否都能在《产品需求规格说明书》中找到出处。

◆正向跟踪和逆向跟踪合称为“**双向跟踪**”。不论采用何种跟踪方式，都要建立与维护**需求跟踪矩阵(即表格)**。需求跟踪矩阵保存了需求与后继工作成果的对应关系。

考试真题

以下（）不属于需求分析的工作

A.分析整理需求 B.逻辑建模

C.可靠性建模

D.需求优先级排序

答案：C

以下关于需求跟踪的说法中，错误的是（）

A. 正向跟踪是检查设计文档、代码、测试用例等工作成果是否都能在《产品需求规格说明书》中找到出处。

B.需求跟踪有两种方式，分别是正向跟踪和逆向跟踪

C. 需求跟踪矩阵保存了需求与后继工作成果的对应关系。

D. 需求跟踪的目的是建立与维护“需求，设计—编程—测试”之间的一致性

答案：A

结构化分析

◆系统分析阶段是应用系统思想和方法，把复杂的对象分解为简单的组成部分，找出这些部分的基本属性和彼此之间的关系的過程，其基本任务是系统分析師和用户在充分了解用户需求的基础上，把双方对新系统的理解表达为系统需求规格说明书。

系统设计的目标是根据系统分析的结果，完成系统的构建过程。其主要目的是绘制系统的蓝图，权衡和比较各种技术和实施方法的利弊，合理分配各种资源，构建新系统的详细设计方案和相关模型，指导系统实施工作的顺利开展。系统设计的主要内容包括概要设计和详细设计。

◆结构化分析方法给出一组**帮助系统分析人员产生功能规约的原理与技术**。它一般利用图形表达用户需求，使用的手段主要有**数据流图、数据字典、结构化语言、判定表以及判定树**等。

◆结构化分析的步骤如下：

(1) 分析业务情况，做出**反映当前物理模型的数据流图(DFD)**；

(2) 推导出**等价的逻辑模型的DFD**；

(3) 设计**新的逻辑系统**，生成数据字典和基元描述；

(4) 建立人机接口，**提出可供选择的目標系统物理模型的DFD**；

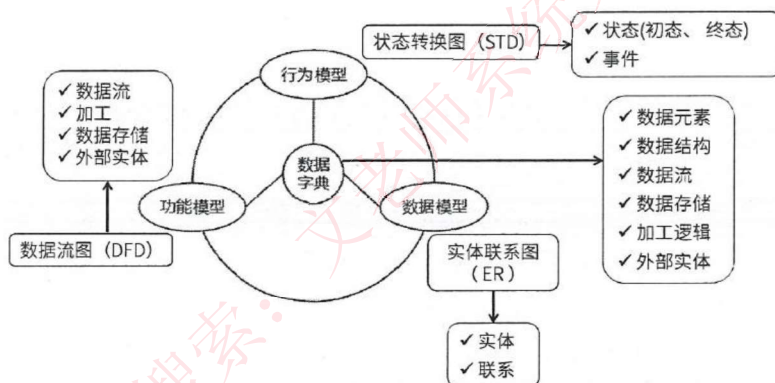
(5) 确定各种方案的成本和风险等级，据此**对各种方案进行分析**；

(6) **选择一种方案**；

(7) **建立完整的需求规约**。

结构化分析

◆三大模型：功能模型（数据流图）、行为模型（状态转换图）、数据模型（E-R图）以及数据字典。



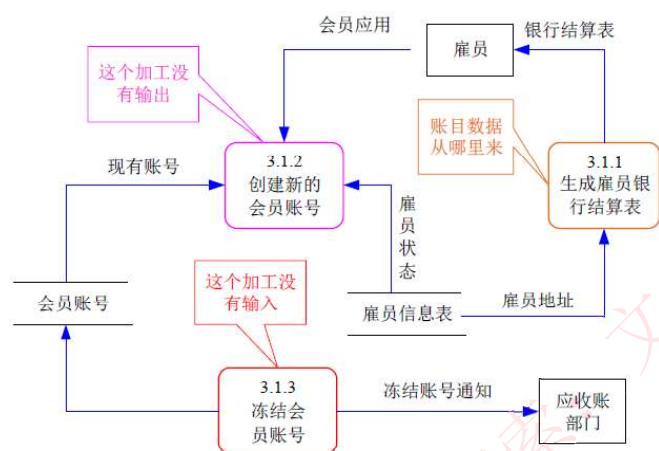
结构化分析

◆数据流图：DFD 需求建模方法，也称为**过程建模和功能建模方法**。DFD 建模方法的核心是**数据流**，从应用系统的数据流着手以图形方式刻画和表示一个具体业务系统中的数据处理过程和**数据流**。

◆ DFD 方法由4种基本元素（模型对象）组成：**数据流、处理 / 加工、数据存储和外部项**。



结构化分析



1) 数据流：用一个箭头描述数据的流向，箭头上标注的内容可以是信息说明或数据项。

2) 处理：表示对数据进行的加工和转换。指向处理的数据流为该处理的输入数据，离开处理的数据流为该处理的输出数据。数据流图中常见的三种错误如图所示：

加工3.1.2 有输入但是没有输出，称之为“黑洞”。

加工3.1.3 有输出但没有输入。称之为“奇迹”。

加工3.1.1 中输入不足以产生输出，我们称之为“灰洞”。

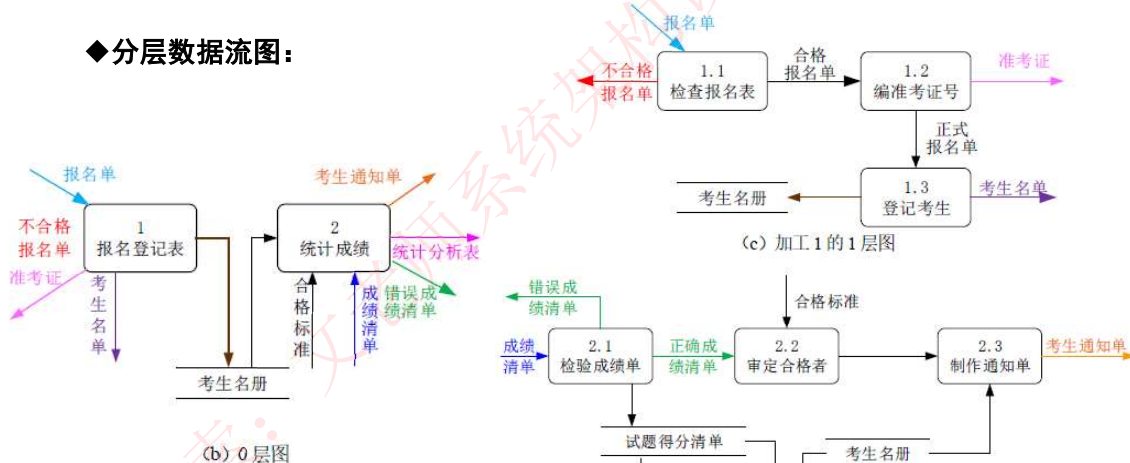
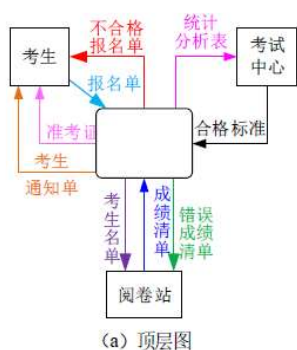
◆这有几种可能的原因：一个错误的命名过程；错误命名的输入或输出；不完全的事实。灰洞是最常见的错误，也是最使人为难的错误。一旦数据流图交给了程序员，到一个加工的输入数据流必须足以产生输出数据流。

3) 数据存储：表示用数据库形式（或者文件形式）存储的数据。

4) 外部项：也称为数据源或者数据终点。描述系统数据的提供者或者数据的使用者，如教师、学生、采购员、某个组织或部门或其他系统。

结构化分析

◆分层数据流图：



结构化分析

◆具体的建模过程及步骤如下：

- (1) **明确目标，确定系统范围。**
- (2) **建立顶层DFD图。**顶层DFD图表达和描述了将要实现的系统的主要功能，同时也确定了整个模型的内外关系，表达了系统的边界及范围。
- (3) **构建第一层DFD分解图。**
- (4) **开发DFD层次结构图。**分解可采用以下原则：保持均匀的模型深；按困难程度进行选择；如果一个处理难以确切命名，可以考虑对它重新分解。
- (5) **检查确认DFD图。**具体规则包括：**父图中描述过的数据流必须要在相应的子图中出现；一个处理至少有一个输入流和一个输出流；一个存储必定有流入的数据流和流出的数据流；一个数据流至少有一端是处理端；**模型图中表达和描述的信息是全面的、完整的、正确的和一致的。

◆数据字典DD：是一种**用户可以访问的记录数据库和应用程序元数据的目录**。数据字典是指对数据的数据项、数据结构、数据流、数据存储、处理逻辑等进行定义和描述，其目的是对数据流程图中的各个元素做出详细的说明。简而言之，数据字典是**描述数据的信息集合**，是对系统中使用的所有数据元素定义的集合。数据字典各部分的描述如下。

- (1) **数据项**：数据流图中数据块的数据结构中的数据项说明。数据项是不可再分的数据单位。
- (2) **数据结构**：数据流图中数据块的数据结构说明。数据结构反映了数据之间的组合关系。
- (3) **数据流**：数据流图中流线的说明。
- (4) **数据存储**：数据流图中数据块的存储特性说明。
- (5) **处理过程**：数据流图中功能块的说明。

结构化设计

◆结构化设计是一种**面向数据流的设计方法**，它以SRS和SA阶段所产生的数据流图和数据字典等文档为基础，是一个**自顶向下、逐步求精和模块化**的过程。SD方法的基本思想是**将软件设计成由相对独立且具有单一功能的模块组成的结构**，分为**概要设计和详细设计**两个阶段。

◆其中概要设计的主要任务是**确定软件系统的结构，对系统进行模块划分，确定每个模块的功能、接口和模块之间的调用关系（输出模块结构图）**；

◆详细设计的主要任务是**为每个模块设计实现的细节（包括模块内详细算法设计、模块内数据结构设计、数据库的物理设计、代码、输入/输出格式、用户界面等设计）**。

◆设计原则：

- (1) **信息隐藏与抽象**：采用封装技术，将程序模块的实现细节隐藏起来，对外仅暴露接口。
- (2) **模块化**：模块是实现功能的基本单位，它一般具有功能、逻辑和状态3个基本属性。模块的外部特性是指模块的模块名、参数表和给程序乃至整个系统造成的影响，而模块的内部特性则是指完成其功能的程序代码和仅供该模块内部使用的数据。
- (3) **高内聚、低耦合**。

结构化设计

◆内聚程度从低到高如下表所示：

内聚分类	定义	记忆关键字
偶然内聚	一个模块内的各处理元素之间没有任何联系	无直接关系
逻辑内聚	模块内执行若干个逻辑上相似的功能，通过参数确定该模块完成哪一个功能	逻辑相似、参数确定
时间内聚	把需要同时执行的动作组合在一起形成的模块。	同时执行
过程内聚	一个模块完成多个任务，这些任务必须按指定的过程执行	指定的过程顺序
通信内聚	模块内的所有处理元素都在同一个数据结构上操作，或者各处理使用相同的输入数据或者产生相同的输出数据	相同数据结构、相同输入输出
顺序内聚	一个模块中的各个处理元素都密切相关于同一功能且必须顺序执行，前一个功能元素的输出就是下一个功能元素的输入	顺序执行、输入为输出
功能内聚	最强的内聚，模块内的所有元素共同作用完成一个功能，缺一不可	共同作用、缺一不可

结构化设计

◆耦合程度从低到高如下所示：

耦合分类	定义	记忆关键字
非直接耦合	两个模块之间没有直接的关系，它们分别从属于不同模块的控制与调用，不传递任何信息	无直接关系
数据耦合	两个模块之间有调用关系，传递的是简单的数据值，相当于高级语言中的值传递	传递数据值调用
标记耦合	两个模块之间传递的是数据结构	传递数据结构
控制耦合	一个模块调用另一个模块时，传递的是控制变量，被调用模块通过该控制变量的值有选择的执行模块内的某一功能	控制变量、选择执行某一功能
通信耦合	一组模块共用了一组输入信息，或者它们的输出需要整合以形成完整数据，即共享了输入或输出	共用通信信息
公共耦合	通过一个公共数据环境相互作用的那些模块间的耦合	公共数据结构
内容耦合	当一个模块直接使用另一个模块的内部数据，或通过非正常入口转入另一个模块内部时。	模块内部关联

结构化设计

◆系统结构图（SC）又称为**模块结构图**，它是软件概要设计阶段的工具，反映系统的功能实现和模块之间的联系与通信，**包括各模块之间的层次结构，即反映了系统的总体结构。**

◆详细设计的基本步骤：

- （1）分析并确定**输入 / 输出数据的逻辑结构**。
- （2）找出输入数据结构和输出数据结构中有**对应关系的数据单元**。
- （3）按一定的规则**由输入、输出的数据结构导出程序结构**。
- （4）列出基本操作与条件，并把它们**分配到程序结构图的适当位置**。
- （5）**用伪码写出程序**。

◆详细设计的表示工具有**图形工具、表格工具和语言工具**。其中图形工具有**业务流图、程序流程图、PAD图、NS流程图**。

◆程序流程图又称为**程序框图**。用方框表示一个处理步骤，菱形表示一个逻辑条件，箭头表示控制流向。其优点是：**结构清晰，易于理解，易于修改**。缺点是：**只能描述执行过程而不能描述有关的数据**。

◆NS流程图，也称为**盒图**，是一种强制使用结构化构造的图示工具，也称为方框图。其具有以下特点：**功能域明确、不可能任意转移控制、很容易确定局部和全局数据的作用域、很容易表示嵌套关系及模板的层次关系**。

◆PAD图是一种改进的图形描述方式，可以用来取代程序流程图，相比程序流程图更直观，结构更清晰。最大的优点是**能够反映和描述自顶向下的历史和过程**。PAD提供了5种基本控制结构的图示，并允许递归使用。

考试真题

在结构化分析中，用数据流图描述（ ）。当采用数据流图对一个图书馆管理系统进行分析时，（ ）是一个外部实体。

- A. 数据对象之间的关系，用于对数据建模
B. 数据在系统中如何被传送或变换，以及如何对数据流进行变换的功能或子功能，用于对功能建模
C. 系统对外部事件如何响应，如何动作，用于对行为建模
D. 数据流图中的各个组成部分
A. 读者 B. 图书 C. 借书证 D. 借阅

答案：B A

下列选项中，内聚类型排序从高到低的是（ ）

- A. 功能内聚、时间内聚、顺序内聚
B. 顺序内聚、通信内聚、过程内聚
C. 过程内聚、顺序内聚、逻辑内聚
D. 时间内聚、顺序内聚、偶然内聚

答案：B

人机界面设计

◆人机界面设计的三大原则：**置于用户控制之下、减少用户的记忆负担、保持界面的一致性。**

◆置于用户的控制之下

以不强迫用户进入不必要的或不希望的动作的方式来定义交互方式；

提供灵活的交互；

允许用户交互可以被中断和取消；

当技能级别增加时可以使交互流水化并允许定制交互；

使用户隔离内部技术细节；

设计应允许用户和出现在屏幕上的对象直接交互。

◆减少用户的记忆负担

减少对短期记忆的要求；

建立有意义的缺省；

定义直觉性的捷径；

界面的视觉布局应该基于真实世界的隐喻；

以不断进展的方式揭示信息。

◆保持界面的一致性

允许用户将当前任务放入有意义的语境

在应用系列内保持一致性

如过去的交互模型已建立起了用户期望，除非有迫不得已的理由，不要去改变它。

测试方法

◆软件测试是**使用人工或自动的手段来运行或测定某个软件系统的过程**，其目的在于**检验它是否满足规定的需求或弄清预期结果与实际结果之间的差别**。

◆软件测试的**目的就是确保软件的质量、确认软件以正确的方式做了用户所期望的事情**，所以软件测试工作主要是**发现软件的错误**、有效定义和实现软件成分由低层到高层的组装过程、验证软件是否满足任务书和系统定义文档所规定的技术要求、为软件质量模型的建立提供依据。

◆测试方法的分类

◆测试过程中程序执行状态：**动态测试、静态测试**。

◆具体实现算法细节和系统内部结构的相关情况为根据可分**黑盒测试、白盒测试和灰盒测试**。

◆程序执行方式：**人工测试、自动化测试**（自动化执行测试脚本）。

（1）静态测试：静态测试是被测程序不运行，**只依靠分析或检查源程序的语句、结构、过程等来检查程序是否有错误**。即通过对软件的需求规格说明书、设计说明书以及源程序做结构分析和流程图分析，从而来找出错误。例如不匹配的参数，未定义的变量等，其方法包括：

◆桌前检查：**程序员检查自己编写的程序**，在程序编译后，单元测试前。

◆代码审查：由**若干个程序员和测试人员组成评审小组**，通过召开程序评审会来进行审查。

◆代码走查：也是采用开会来对代码进行审查，但并非简单的检查代码，而是**由测试人员提供测试用例，让程序员扮演计算机的角色**，手动运行测试用例，检查代码逻辑。

（2）动态测试：通过运行被测试程序，**对得到的运行结果与预期的结果进行比较分析，同时分析运行效率和健壮性能等**。这种方法可简单分为**3个步骤：构造测试实例、执行程序以及分析结果**。

测试方法

(3) 黑盒测试法：黑盒测试**将被测程序看成是一个黑盒**，工作人员在不考虑任何程序内部结构和特性的条件下，根据需求规格说明书设计测试实例，并检查程序的功能是否能够按照规范说明准确无误的运行。其主要是对**软件界面和软件功能**进行测试。对于黑盒测试行为必须加以量化才能够有效的保证软件的质量。

(4) 白盒测试法：白盒测试主要是**借助程序内部的逻辑和相关信息，通过检测内部动作是否按照设计规格说明书的设定进行，检查每一条通路能否正常工作**。白盒测试是从程序结构方面出发对测试用例进行设计。主要用于检查各个逻辑结构是否合理，对应的模块独立路径是否正常以及内部结构是否有效。常用的白盒测试法有**控制流分析、数据流分析、路径分析、程序变异**等。根据测试用例的覆盖程度，分为**语句覆盖、判定覆盖、分支覆盖和路径覆盖**等。

(5) 灰盒测试法：灰盒测试**介于黑盒与白盒测试之间**。灰盒测试除了**重视输出相对于输入的正确性，也看重其内部的程序逻辑**。但是，它不可能像白盒测试那样详细和完整。它只是**简单地靠一些象征性的现象或标志来判断其内部的运行情况**，因此在内部结果出现错误，但输出结果正确的情况下可以采取灰盒测试方法。因为在此情况下灰盒比白盒高效，比黑盒适用性广的优势就凸显出来了。

(6) 自动化测试。自动化测试就是**软件测试的自动化，即在预先设定的条件下自动运行被测程序，并分析运行结果**。总的来说，这种测试方法就是将人驱动测试行为转化为机器执行的一种过程。

测试阶段

(1) 单元测试：也称为**模块测试**，测试的对象是**可独立编译或汇编的程序模块**、软件构件或OO软件中的类（统称为模块），测试**依据是软件详细设计说明书**。主要是对该软件的模块进行测试，通过测试以发现该模块的功能不符合/不满足期望的情况和编码错误。

◆由于模块的规模不大，功能单一，结构较简单，且测试人员可通过阅读源程序清楚知道其逻辑结构，**首先应通过静态测试方法**，比如静态分析、代码审查等，对该模块的源程序进行分析，按照模块的程序设计的控制流程图，以满足软件覆盖率要求的逻辑测试要求。另外，**也可采用黑盒测试方法提出一组基本的测试用例，再用白盒测试方法进行验证**。若用黑盒测试方法所产生的测试用例满足不了软件的覆盖要求，可采用白盒法增补出新的测试用例，以满足所需的覆盖标准。其所需的覆盖标准应视模块的具体情况而定。对一些质量要求和可靠性要求较高的模块，一般要满足所需条件的组合覆盖或者路径覆盖标准。

(2) 集成测试：目的是**检查模块之间，以及模块和已集成的软件之间的接口关系**，并验证已集成的软件是否符合设计要求。测试**依据是软件概要设计文档**，集成测试通常要对已经严格按照程序设计和标准**组装起来的模块同时进行测试**，明确该程序结构组装的正确性，发现和**接口**有关的问题。在这一阶段，一般采用**白盒测试和黑盒测试结合的方法进行测试**，验证这一阶段设计的合理性以及需求功能的实现性。

测试阶段

(3) 系统测试：一般情况下，系统测试采用**黑盒测试**，以此来检查该系统是否符合软件需求。本阶段的主要测试内容包括**功能测试、性能测试、健壮性测试、安装或反安装测试、用户界面测试、压力测试、可靠性及安全性测试**等。测试人员必须进行**多轮回归测试**。系统测试的结束标志是**测试工作已满足测试目标所规定的需求覆盖率，并且测试所发现的缺陷都已全部归零**。

(4) 性能测试：是通过**自动化的测试工具模拟多种正常、峰值以及异常负载条件来对系统的各项性能指标进行测试**。**负载测试和压力测试**都属于性能测试，两者可以结合进行。通过负载测试，确定在**各种工作负载下系统的性能**，目标是测试当负载逐渐增加时，系统各项性能指标的变化情况。压力测试是通过确定一个系统的瓶颈或者不能接受的性能点，来获得**系统能提供的最大服务级别的测试**。

(5) 验收测试，是最后一个阶段的测试，是软件产品投入正式交付前的测试工作。是要满足用户需求或者与用户签订的合同的各项要求，是由用户对要交付软件开展的一种测试工作。主要目标是为用户展示所开发出来的软件符合预定的要求和有关标准，并验证软件实际工作的有效性和可靠性，确保用户能用该软件顺利完成既定的任务和功能。

通过了验收测试，该产品就可进行发布。从用户的角度出发，测试人员还应进行Alpha 测试或Beta 测试。Alpha 测试是在软件开发环境下由用户进行的测试。而Beta 测试是在实际环境中由多个用户对其进行测试。

(6) 回归测试：测试目的是**测试软件变更之后，变更部分的正确性和对变更需求的符合性，以及软件原有的、正确的功能、性能和其他规定的要求的不损害性**。

测试阶段

◆其他测试

(1) **AB 测试是为Web 或App 界面或流程制作两个或多个版本**，在同一时间维度，分别让组成成分相同（相似）的访客群组（目标人群）随机的访问这些版本，收集各群组的用户体验数据和业务数据，最后分析、评估出最好版本，正式采用。

(2) Web 测试是软件测试的一部分，**是针对Web 应用的一类测试**。由于Web 应用与用户直接相关，又通常需要承受长时间的大量操作，因此Web 项目的功能和性能都必须经过可靠的验证。

(3) 链接测试。链接是Web 应用系统的一个主要特征，它是在页面之间切换和指导用户去一些未知地址页面的主要手段。链接测试可分为3个方面。首先，**测试所有链接是否按指示那样确实链接到了该链接的页面**；其次，**测试所链接的页面是否存在**；最后，**保证Web 应用系统上没有孤立的页面**。

(4) 表单测试。当**用户通过表单提交信息的时候**，都希望表单能正常工作。如果使用表单来进行在线注册，要确保**提交按钮能正常工作，当注册完成后应返回注册成功的消息**。如果使用表单收集配送信息，应确保程序能够正确处理这些数据，最后能让用户收到信息。

考试真题

单元测试的依据是（）

A.概要设计 B.需求分析 C.详细设计 D.用户需求

答案：C

以下（）不属于白盒测试

A.控制流分析 B.数据流分析 C.程序变异测试 D.功能测试

答案：D

要测试复杂程序逻辑应该选择（）方法，（）既考虑了功能需求，又考虑了内部结构

A.性能测试 B.黑盒测试 C.静态测试 D.压力测试
A.黑盒测试 B.白盒测试 C.灰盒测试 D.静态测试

答案：C C

（）是为Web或App界面或流程制作两个或多个版本，在同一时间维度，分别让组成成分相同（相似）的访客群组（目标人群）随机的访问这些版本，收集各群组的用户体验数据和业务数据，最后分析、评估出最好版本，正式采用。

A.web测试 B.链接测试 C.表单测试 D.AB测试

答案：D

测试用例设计

◆**黑盒测试用例**：将程序看做一个黑盒子，只知道输入输出，不知道内部代码，由此设计出测试用例，分为下面几类：

◆**等价类划分**：把所有的数据按照某种特性进行归类，而后在每类的数据里选取一个即可。等价类测试用例的设计原则：**设计一个新的测试用例，使其尽可能多地覆盖尚未被覆盖的有效等价类**，重复这一步，直到所有的有效等价类都被覆盖为止；**设计一个新的测试用例，使其仅覆盖一个尚未被覆盖的无效等价类**，重复这一步，直到所有的无效等价类都被覆盖为止。

◆**边界值划分**：将每类的边界值作为测试用例，边界值一般为**范围的两端值以及在此范围之外的与此范围间隔最小的两个值**，如年龄范围为0-150，边界值为0, 150, -1, 151四个。

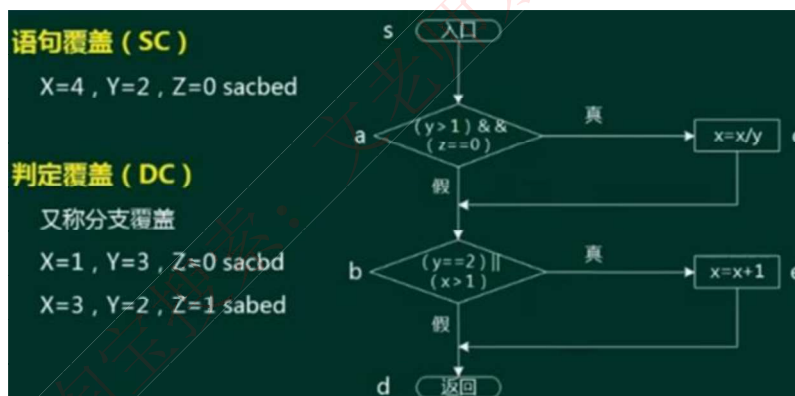
◆**错误推测**：没有固定的方法，凭经验而言，来推测有可能产生问题的地方，作为测试用例进行测试。

◆**因果图**：由一个结果来反推原因的方法，具体结果具体分析，没有固定方法。

测试用例设计

◆白盒测试用例：

- (1) **语句覆盖SC**：逻辑代码中的**所有语句都要被执行一遍**，覆盖层级最低，因为执行了所有的语句，不代表执行了所有的条件判断。
- (2) **判定覆盖DC**：逻辑代码中的**所有判断语句的条件的真假分支都要覆盖一次**。
- (3) **条件覆盖CC**：针对每一个判断条件内的**每一个独立条件都要执行一遍真和假**。
- (4) **条件判定组合覆盖CDC**：**同时满足判定覆盖和条件覆盖**。
- (5) **路径覆盖**：逻辑代码中的**所有可行路径都覆盖了**，覆盖层级最高。



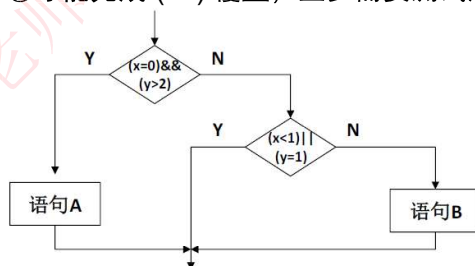
考试真题

招聘系统要求求职的人年龄在20岁到60岁之间（含），学历为本科、硕士或者博士，专业为计算机科学与技术、通信工程或者电子工程。其中（ ）不是好的测试用例。

- A. (20,本科, 电子工程) B. (18,本科, 通信工程)
C. (18, 大专, 电子工程) D. (25, 硕士, 生物学)

答案：C

采用白盒测试方法对下图进行测试，设计了4 个测试用例：①(x=0, y=3), ②(x=1, y=2), ③(x=-1, y=2), ④(x=3, y=1)。至少需要测试用例①②才能完成 (35) 覆盖，至少需要测试用例①②③或①②④才能完成 (36) 覆盖。



- A. 语句 B. 条件
C. 判定 / 条件 D. 路径
A. 语句 B. 条件
C. 判定 / 条件 D. 路径

答案：A D

解析：可根据问题将测试用例代入执行，①②会执行语句A和语句B，但只覆盖第一个条件，因此是语句覆盖；①②覆盖了第一个条件的真假，执行了第一个条件的Y和N以及第二个条件的N，而③或者④又执行到了第二个条件的Y，因此流程中所有分支都走到了，是路径覆盖。

McCabe度量法

◆ McCabe度量法：又称为**环路复杂度**，以程序流程图为基础，有三种计算公式：

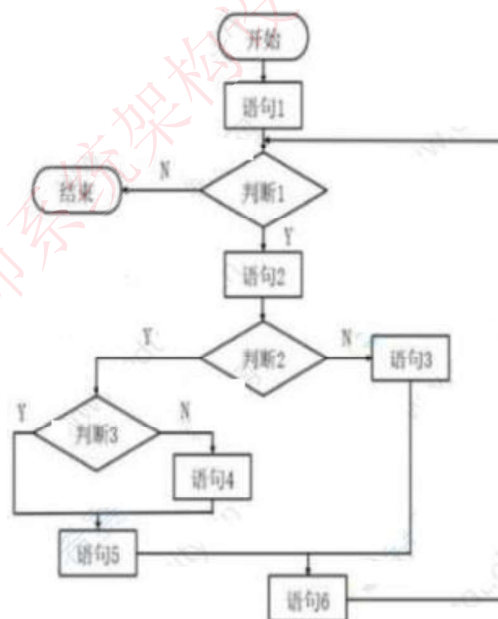
- (1) 边数-顶点数+2
- (2) 封闭区域个数+1
- (3) **判定节点个数+1（推荐使用方法）**

下图用白盒测试方法进行测试，图中有（ ）条路径，采用McCabe度量计算该程序图的环路复杂性为（ ）

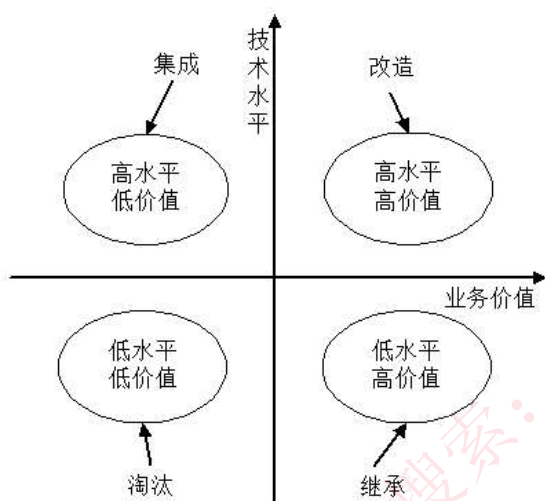
A、3 B、4 C、5 D、6

A、3 B、4 C、5 D、6

答案：B B



系统运行与维护



◆ **遗留系统**是指任何基本上**不能进行修改和演化以满足新的变化了的业务需求的信息系统**，它通常具有以下特点：

(1) 系统虽然完成企业中许多重要的业务管理工作，但仍然**不能完全满足要求**。一般实现业务处理电子化及部分企业管理功能，很少涉及经营决策。

(2) 系统在性能上已经落后，采用的**技术已经过时**。例如，多采用主机/终端形式或小型机系统，软件使用汇编语言或第三代程序设计语言的早期版本开发，使用文件系统而不是数据库。

(3) 通常是大型的软件系统，已经融入企业的业务运作和决策管理机制之中，**维护工作十分困难**。

(4) 没有使用现代信息系统建设方法进行管理和开发，现在基本上已经没有文档，**很难理解**。

系统运行与维护

◆系统转换是指**新系统开发完毕，投入运行，取代现有系统的过程**，需要考虑多方面的问题，以实现与老系统的交接，有以下三种转换计划：

◆**直接转换**：**现有系统被新系统直接取代了**，风险很大，适用于新系统不复杂，或者现有系统已经不能使用的情况。优点是节省成本。

◆**并行转换**：**新系统和老系统并行工作一段时间**，新系统经过试运行后再取代，若新系统在试运行过程中有问题，也不影响现有系统的运行，风险极小，在试运行过程中还可以比较新老系统的性能，适用于大型系统。缺点是耗费人力和时间资源，难以控制两个系统间的数据转换。

◆**分段转换**：**分期分批逐步转换**，是直接和并行转换的集合，**将大型系统分为多个子系统**，依次试运行每个子系统，成熟一个子系统，就转换一个子系统。同样适用于大型项目，只是更耗时，而且现有系统和新系统间混合使用，需要协调好接口等问题。

◆**数据转换与迁移**：将数据从旧数据库迁移到新数据库中。有三种方法：**系统切换前通过工具迁移、系统切换前采用手工录入、系统切换后通过新系统生成**。

系统运行与维护

◆系统的**可维护性**可以定义为**维护人员理解、改正、改动和改进这个软件的难易程度**，其评价指标如下：

- (1) **易分析性**。软件产品诊断软件中的缺陷或失效原因或识别待修改部分的能力。
- (2) **易改变性**。软件产品使指定的修改可以被实现的能力，实现包括编码、设计和文档的更改。
- (3) **稳定性**。软件产品避免由于软件修改而造成意外结果的能力。
- (4) **易测试性**。软件产品使已修改软件能被确认的能力。
- (5) **维护性的依从性**。软件产品遵循与维护性相关的标准或约定的能力。

◆系统维护包括**硬件维护、软件维护和数据维护**，其中**软件维护类型如下**：

正确性维护：发现了bug而进行的修改。

适应性维护：由于外部环境发生了改变，被动进行的对软件的修改和升级。

完善性维护：基于用户主动对软件提出更多的需求，修改软件，增加更多的功能，使其比之前的软件功能、性能更高，更加完善。

预防性维护：对未来可能发生的bug进行预防性的修改。

净室软件工程

◆净室软件工程是一种应用数学与统计学理论以经济的方式生产高质量软件的工程技术，力图通过严格的工程化的软件过程达到开发中的零缺陷或接近零缺陷。净室方法不是先制作一个产品，再去消除缺陷，而是要求在规约和设计中消除错误，然后以“净”的方式制作，可以降低软件开发中的风险，以合理的成本开发出高质量的软件。

◆在净室软件工程背后的哲学是：通过在第1次正确地书写代码增量，并在测试前验证它们的正确性，来避免对成本很高的错误消除过程的依赖。它的过程模型是在代码增量积聚到系统的过程的同时，进行代码增量的统计质量验证。它甚至提倡开发者不需要进行单元测试，而是进行正确性验证和统计质量控制。

◆净室软件工程（CSE）的理论基础主要是函数理论和抽样理论。

◆净室软件工程应用技术手段：

1. 统计过程控制下的增量式开发。
2. 基于函数的规范与设计。
3. 正确性验证。CSE的核心
4. 统计测试和软件认证。

◆净室软件工程在使用过程的一些缺点：

- (1) CSE太理论化，需要更多的数学知识。其正确性验证的步骤比较困难且比较耗时。
- (2) CSE 开发小组不进行传统的模块测试，这是不现实的。
- (3) CSE也会带有传统软件工程的一些弊端。

基于构件的软件工程

◆基于构件的软件工程（CBSE）是一种基于分布对象技术、强调通过可复用构件设计与构造软件系统的软件复用途径。CBSE 体现了“购买而不是重新构造”的哲学，将软件开发的重点从程序编写转移到了基于已有构件的组装。用于CBSE 的构件应该具备以下特征。

(1) 可组装型：对于可组装的构件，所有外部交互必须通过公开定义的接口进行。同时它还必须对自身信息的外部访问。

(2) 可部署性：软件必须是自包含的，必须能作为一个独立实体在提供其构件模型实现的构件平台上运行。构件总是二进制形式，无须在部署前编译。

(3) 文档化：构件必须是完全文档化的，用户根据文档来判断构件是否满足需求。

(4) 独立性：构件应该是独立的，应该可以在无其他特殊构件的情况下进行组装和部署，如确实需要其他构件提供服务，则应显示声明。

(5) 标准化：构件标准化意味着在CBSE 过程中使用的构件必须符合某种标准化的构件模型。构件模型定义了构件实现、文档化以及开发的标准，其包含的模型要素为：

◆接口。构件通过构件接口来定义，构件模型规定应如何定义构件接口以及在接口定义中应该包含的要素，如操作名、参数以及异常等。

◆使用信息。为使构件远程分布和访问，必须给构件一个特定的、全局唯一的名字或句柄。构件元数据是构件本身相关的数据，比如构件的接口和属性信息。

◆部署。构件模型包括一个规格说明，指出应该如何打包构件使其部署成为一个独立的可执行实体。部署信息中包含有关包中内容的信息和它的二进制构成的信息。

基于构件的软件工程

◆ 构件模型提供了一组**被构件使用的通用服务**，这种服务包括以下两种。

· **平台服务**，允许构件在分布式环境下通信和互操作。

· **支持服务**，这是很多构件需要的共性服务。例如，构件都需要的身份认证服务。

中间件实现共性的构件服务，并提供这些服务的接口。

◆ CBSE过程是支持基于构件组装的软件开发过程，过程中的6个主要活动：**系统需求概览、识别候选构件、根据发现的构件修改需求、体系结构设计、构件定制与适配、组装构件创建系统。**

◆ CBSE过程与传统软件开发过程**不同点**：

(1) CBSE 早期**需要完整的需求**，以便尽可能多地识别出可复用的构件。

(2) 在过程早期阶段根据**可利用的构件来细化和修改需求**。如果可利用的构件不能满足用户需求，就应该考虑由复用构件支持的相关需求。

(3) 在系统体系结构设计完成后，会有一个**进一步的对构件搜索及设计精化的活动**。可能需要为某些构件寻找备用构件，或者修改构件以适合功能和架构的要求。

(4) 开发就是将**已经找到的构件集成在一起的组装过程**。

基于构件的软件工程

◆ 构件组装是指构件相互直接集成或是用专门编写的“**胶水代码**”将它们整合在一起来创建一个系统或另一个构件的过程。常见的组装构件有以下3种组装方式。

1. **顺序组装**。通过**按顺序调用已经存在的构件**，可以用两个已经存在的构件来创建一个新的构件。如上一个构件输出作为下一个构件的输入。

2. **层次组装**。这种情况发生在一个构件**直接调用自另一个构件所提供的服务时**。被调用的构件为调用的构件提供所需的服务。二者之间接口匹配兼容。

3. **叠加组装**。这种情况发生在**两个或两个以上构件放在一起创建一个新构件的时候**。这个新构件合并了原构件的功能，从而对外提供了新的接口。外部应用可以通过新接口来调用原有构件的接口，而原有构件不互相依赖，也不互相调用。这种组装类型适合于构件是程序单元或者构件是服务的情况。

◆ 构件组装的**三种不兼容问题**（通过编写适配器解决）：

(1) **参数不兼容**。接口每一侧的操作有相同的名字，但参数类型或参数个数不相同。

(2) **操作不兼容**。提供接口和请求接口的操作名不同。

(3) **操作不完备**。一个构件的提供接口是另一个构件请求接口的一个子集，或者相反。

考试真题

系统上线运行后，用户提出了新的需求，需要进行系统改造升级，提升系统性能，提高运行效率，这属于（）

- A.预防性维护 B.改正性维护 C.完善性维护 D.适应性维护
答案：C

净室软件工程的理论基础主要是（）

- A.函数理论和抽样理论 B.概率统计和软件认证
C.统计测试和正确性验证 D.质量保证和概率统计
答案：A

以下关于构建组装的说法中正确的是（）

- A.构件组装方式包括顺序组装、层次组装、循环组装三种
B.构件组装会存在参数不兼容、操作不兼容这两种兼容性问题
C.构件组装操作需要完备才能保证系统功能
D.层次组装发生在两个或两个以上构件放在一起创建一个新构件的时候
答案：C

谢谢！

文老师软考教育